Structural Bioinformatics with BiochemicalAlgorithms.jl

1st author[1], 2nd author[1], and 3rd author[1]

[1]Scientific Computing and Bioinformatics, Institute of Computer Science, Johannes Gutenberg-University Mainz

ABSTRACT

General Terms

Experimentation, Human Factors

Keywords

Julia, Structural Bioinformatics, BALL, C++

1. Introduction

The aim of structural bioinformatics is the analysis and targeted manipulation of three dimensional structures of biological macromolecules such as proteins and nucleic acids (RNA/DNA). This research area combines and applies knowledge from diverse disciplines ranging from fundamental physical laws such as Newton's equation of motion to complex situations requiring in-depth knowledge of biochemistry and advanced numerical computing methodologies.
More precisely, molecular modelling techniques typically rely on molecular mechanics in which a molecular force fields is being used to compute the energy of the examining structure. Resulting applications include structure minimization, molecular dynamics (MD) simulations and molecular docking scenarios. The latter is inevitable for drug design because it computes the best configuration for a given protein and a ligand (the drug) and compute the binding affinity. The importance of molecular applications has been demonstrated during the Covid-19 pandemic. Tools for molecular modelling and in particular docking suites were attracting widespread interest again in order to model SARS-COV-2 proteins and running molecular docking simultions.

Write about: Rational Drug Design economically important

For more than 50 years researcher have been studying molecular functions of proteins. The first attempts date back to 1959, when Max Perutz and John Kendrew used X-ray christallography to to solve the structure of hemoglobine.
A vital aspect of this research field is the availability of molecular structure data, which was solved by the initialization of the protein data bank in 1972. Although it only contained 7 structures, it now provides 227.561 experimentally solved structures (dated on:). With the rapid rise of computed structures

check number of structures in pdb

Software packages for handling molecular structures and molecular applications are available for many years and can be rougly divided into open-source and closed-sourced tools. Most software packages were created from 1995 to 2010 and written in C++. Although this choice of programming language enables the required efficiency, it does not allow for rapid prototyping, which is why some software packages provide an additional interface in a scripting language like Python.
The tools were often only designed for one specific task e.g., implementation of a structure minimization algorithm, docking algorithm... One example for the latter is Schroedinger

Cite

—docking algorithmus

cite single approache

—kraftfelder..

cite schroed

Why do we have to run simulations on the computer? Why is it necessary to perform it on a computer? Example Protein Ligand Docking... As an example.
There remains a need for a basis from which
Here, we present BiochemicalAlgorithms.jl . We provide the basis for interesting analysis encompassing the entire molecular modelling pipeline:

—Reading common data formats such as PDB, hin, mol and JSON
—Preprocessing the input by preparing the entire system ready to simulate.
—Molecular Mechanics such as AMBER ForceField
—(Output writing) such as JSON

BiochemicalAlgorithms.jl is designed to be a platform from which other packages can be included.

2. BALL - Biochemical Algorithms Library

Software development is typically challenging in interdisciplinary research fields.

2.1 Design Goals

Find out source

2.2 Structure

cite ball-project homepa Project/

3. A fresh approach for Structural Bioinformatics: BiochemicalAlgorithms.jl

3.1 Why Julia?

3.2 The core

3.3 The internal representation

cite ball-project homepage

When working with structural data of molecular entities such as proteins, or nucleic acid molecules (DNA/RNA), the representation of three dimensional coordinates play an important role. Just like in BALL we define a similar hierarchical structure...as can be seen in fig.. Optimized for random access to any atoms... same hand easy to use for Julia users or Python users familiar with

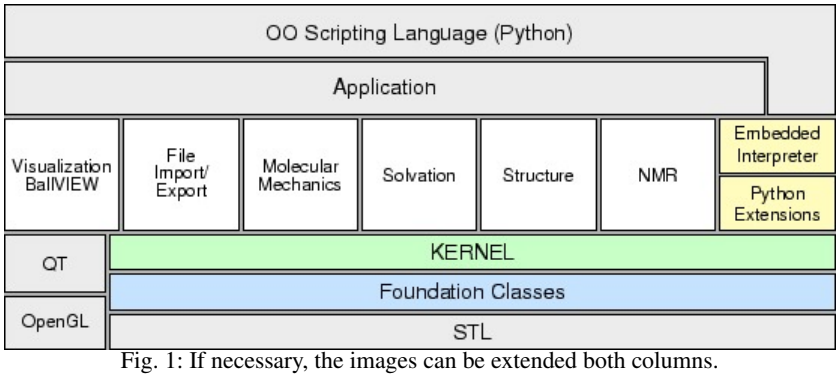

Fig. 1: If necessary, the images can be extended both columns.

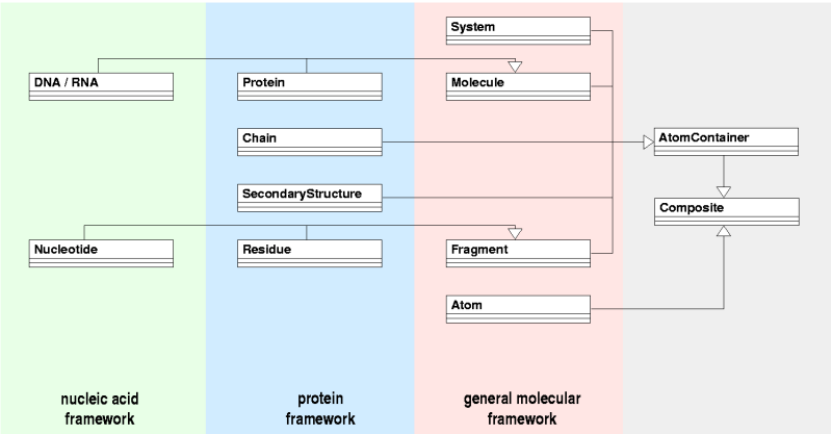

Fig. 2: BALL 's architecture is structured in several layers. Upon the standard libary layer are the foundation classes and ontop of them the kernel. Several module extend the interface for visualization, file import and expoert, molecular mechanics, solvation, structure and NMR. The C++ written frameworkis extended by Python interface for fast scripting purposes. The figure was taken from: https://github.com/BALL-Project/ball/tree/master/doc/doxygen/images

Pandas DataFrames... So initially, representation was based on DataFrames. While theis is extremely intuitive, it has some

3.4 Benchmarks

On par with its C++ predecessor it is intuitiv to write code that is

4. Applications

A trivial comparison between C++ and Julia

4.1 Water molecule

As shown in the code example Code listing 1 the interaction with core components of BiochemicalAlgorithms.jl is straightforward.

Code 1: Intuitive usage of BiochemicalAlgorithms.jl core components

```
using BiochemicalAlgorithms
using BiochemicalVisualization

sys = System()
h2o = Molecule(sys)

o1 = Atom(h2o, 1, Elements.O)
h1 = Atom(h2o, 2, Elements.H)
h2 = Atom(h2o, 3, Elements.H)

h1.r = [1, 0, 0]
h2.r = [cos(deg2rad(105)), sin(deg2rad(105)), 0]

Bond(h2o, o1.idx, h1.idx, BondOrder.Single)
Bond(h2o, o1.idx, h2.idx, BondOrder.Single)

println("Number of atoms: ", natoms(h2o))
println("Number of bonds: ", nbonds(h2o))

ball_and_stick(sys)
stick(sys)
van_der_waals(sys)
```

4.2 RMSD computation and Application of Amber

Code 2: Comparison and mapping of two similar structures

```
sys = load_pdb.(["data/arnd1.pdb",
                 "data/arnd2.pdb"])

fdb = FragmentDB()
normalize_names!.(sys, Ref(fdb))
reconstruct_fragments!.(sys, Ref(fdb))
build_bonds!.(sys, Ref(fdb))

println.(sys)
```

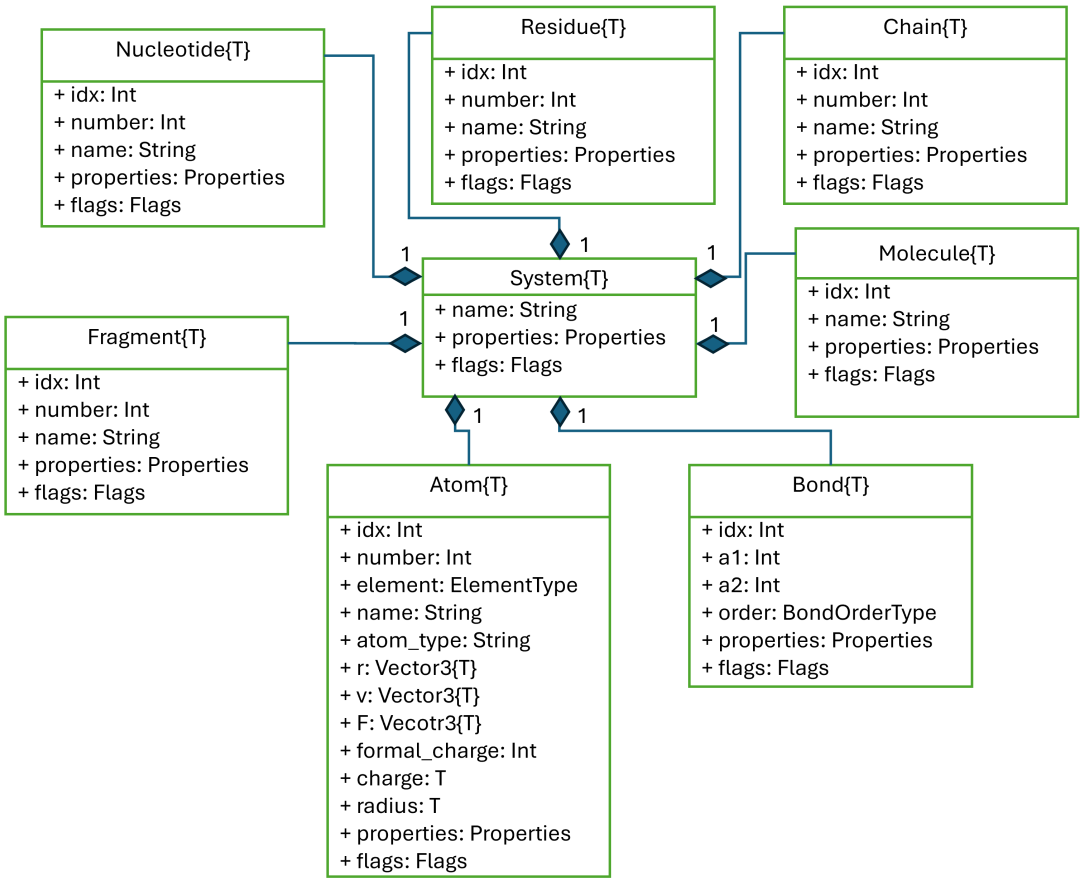

Fig. 3: UML-Diagramm of the core of BiochemicalAlgorithms.jl . In the center resides the `System` interface. All other functionalities are grouped around that core piece. Only the most important functionalities of each class are shown.

```
10
11 compute_energy.(AmberFF.(sys), verbose=true)
12
13 println("RMSD before mapping: ",
14         compute_rmsd(sys[1], sys[2]))
15
16 map_rigid!(sys[1], sys[2])
17
18 println("RMSD after mapping: ",
19         compute_rmsd(sys[1], sys[2]))
```

4.3 Visualization using BiochemicalVisualization.jl

A key feature of BiochemicalAlgorithms.jl is the visualization tool *BiochemicalVisualization.jl* that has been developed alongside the main framework. As can be seen in Figure 4 BiochemicalVisualization.jl currently supports three different representation of protein structures, namely `ball-and-stick`, `van-der-Waals` and `stick` (cf. code listing 1).
When dealing with three dimensional structures of macromolecules, visualization plays an important role for supporting the development of insights into molecular functions. The possibility to visualize and interactively modify the representations provides great support during modelling scenarios. For instance, the tool has been used to visualize different steps from code listing 2. The first image left represent the raw input read from the underlying pdb file, the image in the middle shows the same input after preprocession it with the fragment data base (lines4-7). Finally, the mapping of both structures is shown in the image on the left. As can be seen, the structures do not match perfectly onto each other.
Even this rather simple example already demonstrates the advantage of a visual representation that can be modified and manipulated in context of modelling scenarios and how the visualization supports the development of knowledge of molecular functions.

5. Integration into Julia Universe

6. Conclusion

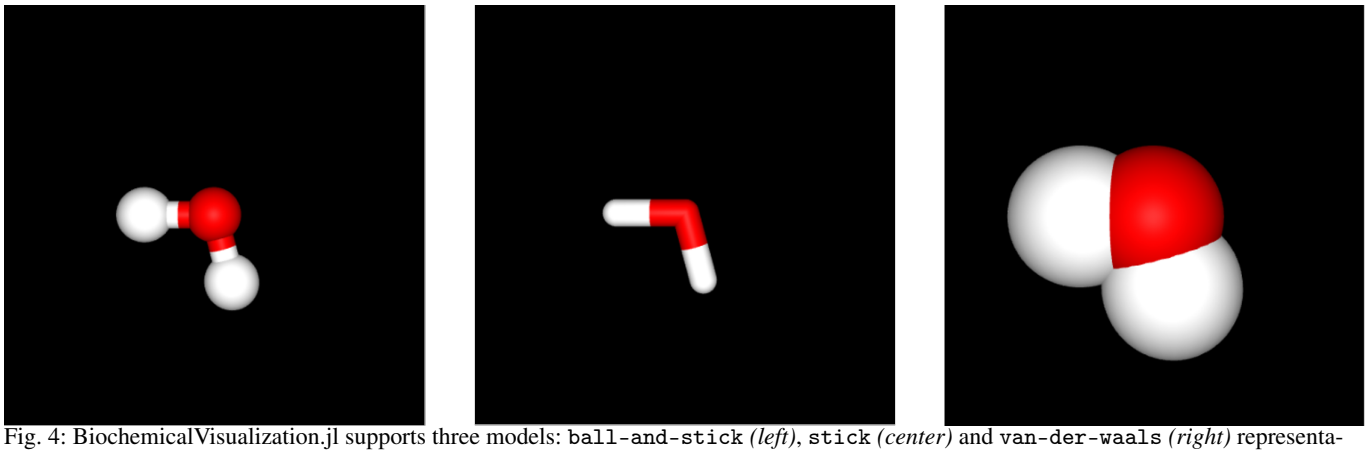

Fig. 4: BiochemicalVisualization.jl supports three models: `ball-and-stick` *(left)*, `stick` *(center)* and `van-der-waals` *(right)* representation of the water molecule as generated by the code listing 1.

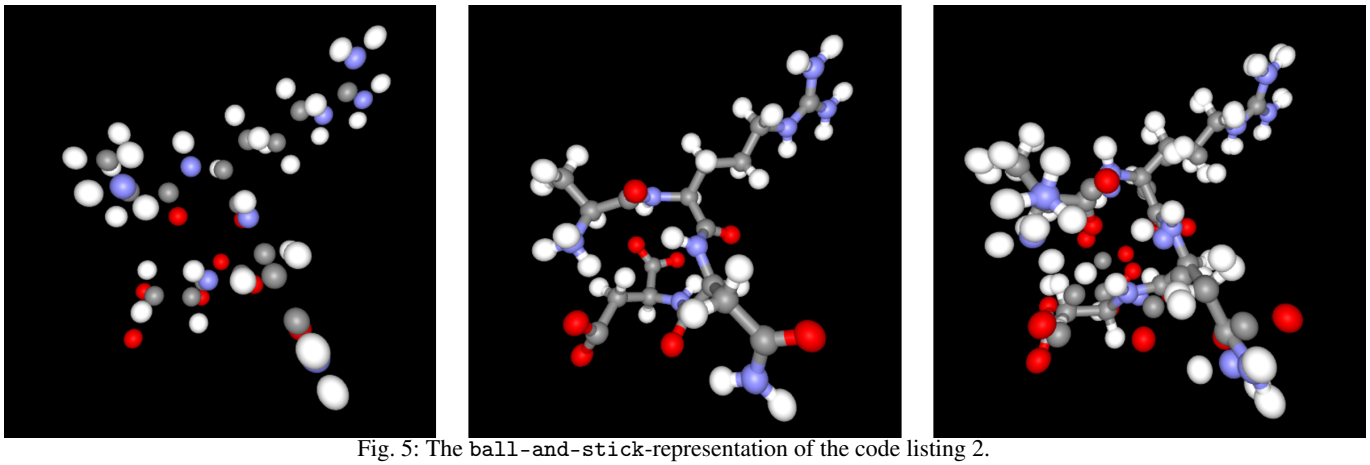

Fig. 5: The `ball-and-stick`-representation of the code listing 2.